

PUC-MINAS
Teste de Software

Refatoração de Testes e Detecção de Test Smells

Aluno: Arthur Astolfi Cardoso
Matrícula: 849022
Repositório: <https://github.com/ArthurAstolfi/test-smelly>

1. Introdução

Este relatório descreve a análise e a refatoração da suíte de testes do projeto **test-smelly**, que simula um serviço de gerenciamento de usuários. A suíte original (`test/userService.smelly.test.js`) foi escrita propositalmente com Test Smells: ela passa no `npm test`, mas tem pouca capacidade de detectar bugs. O trabalho combinou a inspeção manual do código com a análise estática por meio do **ESLint** e do plugin **eslint-plugin-jest**. Em seguida, foi criada uma nova suíte, `test/userService.clean.test.js`, que segue o padrão Arrange, Act, Assert (AAA).

2. Análise de Smells

A análise manual encontrou sete problemas na suíte original. Os três descritos a seguir foram considerados os mais graves.

2.1 Lógica Condicional no Teste (Conditional Test Logic)

Onde: no teste “deve desativar usuários se eles não forem administradores”, que percorre os usuários com um `for` e decide com um `if/else` qual `expect` executar.

Por que é um mau cheiro: um teste deve ser um roteiro linear e determinístico. Com laços e condicionais, o próprio teste passa a ter lógica, e essa lógica também pode ter erros. Além disso, o teste junta dois cenários diferentes (usuário comum e administrador) e não deixa claro qual comportamento está sendo verificado.

Risco: como os `expect` ficam dentro do `if`, eles podem nunca ser executados. Se a lista ficar vazia ou o campo `isAdmin` vier com valor errado, o teste passa sem verificar nada (falso positivo).

2.2 Tratamento de Exceção Silencioso (try/catch com expect condicional)

Onde: no teste “deve falhar ao criar usuário menor de idade”, em que o `expect` fica dentro do bloco `catch`.

Por que é um mau cheiro: o teste só faz a verificação se uma exceção for lançada. Se nenhuma exceção ocorrer, o bloco `catch` não roda, nenhum `expect` é executado e o Jest considera o teste aprovado.

Risco: se alguém remover a validação de idade do `createUser`, o bug passa sem ser notado e o teste continua verde. Um mutante desse tipo sobreviveria ao Teste de Mutação, que foi o tema do trabalho anterior.

2.3 Teste Frágil / Igualdade Sensível (Fragile Test)

Onde: no teste “deve gerar um relatório de usuários formatado”, que compara a linha exata `ID: ${id}, Nome: Alice, Status: ativo\n` e o cabeçalho literal do relatório.

Por que é um mau cheiro: o teste fica preso a detalhes de apresentação (espaços, vírgulas, quebras de linha e ordem dos campos) em vez de verificar o comportamento, que é o relatório conter os dados dos usuários. O próprio código-fonte avisa que “o formato do relatório pode mudar no futuro”.

Risco: qualquer mudança visual sem relação com bugs quebra o teste. Isso gera falsos negativos, aumenta o custo de manutenção e leva a equipe a desconfiar da suíte ou a apagar testes.

2.4 Outros smells identificados

Smell	Ocorrência
Eager Test	Um único teste cria e busca o usuário, com dois “Act” e asserts intercalados.
Disabled / Empty Test	test.skip sem corpo dá falsa sensação de cobertura do cenário “nenhum usuário”.
Valores mágicos / falta de AAA	Literais como 30, 40 e true (isAdmin) aparecem soltos, e as fases do teste estão misturadas.

3. Processo de Refatoração

O teste escolhido para mostrar a refatoração foi o de desativação de usuários, porque reúne três smells ao mesmo tempo: lógica condicional, teste ansioso (dois cenários em um) e valores mágicos.

Antes (userService.smelly.test.js)

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);
  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      expect(resultado).toBe(false);
    }
  }
});
```

Depois (userService.clean.test.js)

```
describe('deactivateUser', () => {
  test('deve desativar um usuário comum e retornar true', () => {
    // Arrange
    const usuarioComum = criarUsuario({ isAdmin: false });

    // Act
    const resultado = userService.deactivateUser(usuarioComum.id);

    // Assert
    expect(resultado).toBe(true);
    expect(userService.getUserById(usuarioComum.id).status).toBe('inativo');
  });

  test('não deve desativar um administrador e deve retornar false', () => {
    // Arrange
    const usuarioAdmin = criarUsuario({ isAdmin: true });

    // Act
    const resultado = userService.deactivateUser(usuarioAdmin.id);

    // Assert
    expect(resultado).toBe(false);
    expect(userService.getUserById(usuarioAdmin.id).status).toBe('ativo');
  });
});
```

Decisões tomadas:

- **Remoção do for e do if/else:** cada cenário virou um teste próprio. Todos os expect agora são incondicionais e sempre executam, o que eliminou os erros jest/no-conditional-expect.
- **Um comportamento por teste:** o caso “usuário comum” e o caso “administrador” falham de forma independente, e o nome do teste já diz o que quebrou.
- **Padrão AAA explícito:** os comentários // Arrange, // Act e // Assert separam preparação, ação e verificação, e há uma única ação por teste.
- **Fim dos valores mágicos:** a função auxiliar criarUsuario({ isAdmin: true }) usa parâmetros nomeados. Isso deixa explícito o que importa para o cenário e esconde dados irrelevantes.
- **Asserção mais forte para o admin:** o teste refatorado também verifica que o status do administrador continua ativo. O teste original verificava apenas o retorno false.

Os demais smells foram corrigidos da mesma forma. O try/catch foi trocado por expect(() => criarUsuario({ idade: 17 })).toThrow(...), que falha se a exceção não for lançada. O teste do relatório passou a verificar se o texto **contém** o id, o nome e o status de cada usuário, em vez da string exata. O teste ansioso foi dividido em testes de createUser e de getUserById. O teste desabilitado foi implementado e verifica a mensagem “Nenhum usuário cadastrado”. Também foram incluídos casos de borda: idade exatamente 18, campos obrigatórios ausentes e id inexistente. A nova suíte tem 15 testes pequenos e focados, agrupados por método com describe.

4. Relatório da Ferramenta (ESLint)

O ESLint foi configurado com o arquivo .eslintrc.json definido no enunciado, que estende eslint:recommended e plugin:jest/recommended. Foi usada a versão 8 do ESLint porque a versão 9 não lê mais o formato .eslintrc.json. A saída da primeira execução sobre o projeto foi:

```
PS C:\Users\Arthur\Faculdade\testeSmell\test-smelly> npx eslint test/userService.smelly.test.js
C:\Users\Arthur\Faculdade\testeSmell\test-smelly\test\userService.smelly.test.js
 44:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 46:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 49:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 73:7  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 77:3  warning Tests should not be skipped        jest/no-disabled-tests
 77:3  warning Test has no assertions          jest/expect-expect

X 6 problems (4 errors, 2 warnings)
PS C:\Users\Arthur\Faculdade\testeSmell\test-smelly>
```

Linha	Regra	Smell correspondente
44, 46, 49	jest/no-conditional-expect	Lógica condicional (seção 2.1)
73	jest/no-conditional-expect	try/catch silencioso (seção 2.2)
77	jest/no-disabled-tests	Teste desabilitado
77	jest/expect-expect	Teste vazio, sem asserções

A ferramenta automatizou a detecção dos smells estruturais, que podem ser reconhecidos pela árvore sintática do código: `expect` dentro de `if` ou `catch`, testes com `skip` e testes sem asserção. Ela aponta o arquivo, a linha e a regra violada em segundos e pode ser executada em todo commit ou pipeline de CI, sem depender da atenção de um revisor. Por outro lado, o ESLint não detectou o Teste Frágil, o Eager Test nem os valores mágicos. Esses smells são semânticos: dependem de entender a intenção do teste. Por isso, a análise estática complementa a revisão humana, mas não a substitui.

5. Validação Final

- `npx eslint test/userService.clean.test.js` → **0 erros e 0 avisos**.
- `npm test` → **2 suítes aprovadas, 19 testes passando** (4 do arquivo original e 15 do novo) e 1 teste ignorado, que é o `test.skip` do arquivo original.

6. Conclusão

O trabalho mostrou que uma suíte “verde” não é necessariamente uma suíte confiável. Os testes originais passavam, mas alguns deles nunca falhariam diante de um bug real. Testes limpos, com uma única responsabilidade, sem lógica condicional, estruturados em AAA e focados em comportamento, funcionam como documentação executável do sistema: quando falham, apontam exatamente o problema, e quando o código muda por motivos legítimos, não quebram sem necessidade.

As ferramentas de análise estática como o ESLint tornam esse cuidado contínuo e barato. Elas aplicam automaticamente regras que uma pessoa poderia esquecer e impedem que novos smells entrem no código. Junto com a revisão humana, que identifica os problemas semânticos que o linter não vê, essas práticas reduzem o custo de manutenção, aumentam a confiança nas refatorações e contribuem para a qualidade e a sustentabilidade do projeto ao longo do tempo.